Step-by-Step Guide: Live Incident Reconstruction

Live incident reconstruction is the process of piecing together what an attacker actually did on a system by correlating the live evidence captured during the incident. Unlike memory or disk forensics, which analyse a snapshot taken after the fact, this method works from the real-time logs and traffic that were collected while the attack was happening.

In the CREST CPIA lab, the incident is generated for you by a scripted attacker. Your job is to reconstruct the four-phase attack from three sources of live evidence and write it up as a coherent timeline.

---

1. The incident you are investigating

The CPIA lab victim (`cpia-victim`) is hit by a scripted attacker. The attack runs in four distinct phases:

Phase Technique Evidence 1	SSH brute force (8 usernames)	`auth.log` (failed logins) + `network.pcap`
2	HTTP reconnaissance (10 paths)	`httpd.log` + `network.pcap`
3	HTTP exploits (path trav, SQLi)	`httpd.log` + `network.pcap`
4	C2-style beacon (5 POSTs)	`httpd.log` + `network.pcap`

Three evidence files hold the story:

- `/work/network.pcap` - the raw packet capture
- `auth.log` - SSH authentication attempts (on the victim)
- `httpd.log` - HTTP requests served (on the victim)

The attacker uses the same evidence you are about to analyse, so the source of truth is the capture and the logs.

---

2. Set up your working environment

Enter the analyst container from the host:

```
docker exec -it cpia-forensics bash
```

Confirm the evidence is present:

```
ls -la /work/network.pcap
ls -la /artifacts/auth.log /artifacts/httpd.log 2>/dev/null
```

> *In a fresh lab the auth.log and httpd.log live inside the victim container and are pulled into the analyst workspace by the reset scenario. The* `network.pcap` *is always at* `/work/network.pcap`*.*

---

3. Establish the baseline with the capture

Start by getting the shape of the traffic. How many packets, and what protocols dominate?

```
tshark -r /work/network.pcap 2>/dev/null | wc -l
tshark -r /work/network.pcap -q -z io,phs 2>/dev/null
```

For the CPIA incident this shows SSH and HTTP as the two dominant protocols - a strong hint that the attack has a network-service phase (SSH) and a web phase (HTTP).

Identify the two parties involved:

```
tshark -r /work/network.pcap -q -z endpoints,ip 2>/dev/null
```

You should see two IPv4 endpoints: the attacker and the victim.

---

4. Reconstruct Phase 1: the SSH brute force

The attacker tries eight usernames in quick succession. In the packet capture, the SSH handshakes are visible as repeated connection attempts to port 22.

List the SSH connections and their source ports:

```
tshark -r /work/network.pcap -Y "tcp.port == 22" -T fields \
       -e frame.number -e ip.src -e ip.dst -e tcp.srcport 2>/dev/null | head
```

Correlate with the authentication log, which records each failed login. Search for the repeated authentication failures:

```
grep -iE "Failed password|invalid user" /artifacts/auth.log 2>/dev/null | head
```

The usernames attempted - `root`, `admin`, `test`, `oracle`, `mysql`, `postgres`, `billy`, `admin123` - are the classic brute-force wordlist. None succeeds; the log shows a failure for each attempt.

Record the timeline start: the first failed SSH login marks the beginning of the incident.

---

5. Reconstruct Phase 2: HTTP reconnaissance

Immediately after the SSH phase, the attacker switches to web probing. List every HTTP request the victim served:

```
tshark -r /work/network.pcap -Y "http.request" -T fields \
       -e frame.number -e ip.src -e http.request.method -e http.request.uri 2>/dev/null
```

The requests reveal the recon path:

```
GET /admin
GET /admin/index.php
GET /config.php
GET /backup
GET /backup.zip
GET /phpmyadmin/
GET /wp-login.php
GET /server-status
GET /robots.txt
GET /files/1.txt
```

Each one is a probe for a known sensitive path: admin panels, configuration files, backup archives, database front-ends, and server status. Cross-check against the web server log:

```
grep -E "GET" /artifacts/httpd.log 2>/dev/null | head
```

---

6. Reconstruct Phase 3: the exploit attempts

Phase 3 is where the attacker stops probing and tries to break in. Three specific exploit techniques are used.

6a. Path traversal

The attacker tries to read `/etc/passwd` by escaping the web root:

```
tshark -r /work/network.pcap -Y "http.request.uri contains \"etc/passwd\"" \
       -T fields -e frame.number -e ip.src -e http.request.uri 2>/dev/null
```

This is the classic `../../` traversal - the guide here shows the exact request:

```
tshark -r /work/network.pcap -Y "http.request" -T fields \
       -e frame.number -e http.request.uri 2>/dev/null | grep -i passwd
```

6b. SQL injection on the login form

The attacker POSTs a SQLi payload to the login endpoint:

```
tshark -r /work/network.pcap -Y "http.request.method == POST" \
       -T fields -e frame.number -e ip.src -e http.request.uri 2>/dev/null
```

The payload uses the `' OR 1=1--` pattern, a universal authentication-bypass probe:

```
tshark -r /work/network.pcap -Y "frame contains \"OR 1=1\"" \
       -T fields -e frame.number 2>/dev/null
```

6c. Malicious file upload probe

The attacker attempts to upload a file named `shell.php` to the `/upload` endpoint. Find the POST:

```
tshark -r /work/network.pcap -Y "frame contains \"shell.php\"" \
       -T fields -e frame.number -e ip.src 2>/dev/null
```

This is a web-shell delivery attempt. Even if it is blocked, it is logged and is a high-severity indicator.

---

7. Reconstruct Phase 4: the C2-style beacon

The final phase is command-and-control style beaconing. The attacker POSTs to a path that looks like a legitimate tracking pixel but carries data:

```
tshark -r /work/network.pcap -Y "http.request.uri contains \"pixel.gif\"" \
       -T fields -e frame.number -e ip.src -e http.request.uri 2>/dev/null
```

This reveals the beacon pattern:

```
POST /pixel.gif?id=beacon&c=1
POST /pixel.gif?id=beacon&c=2
POST /pixel.gif?id=beacon&c=3
POST /pixel.gif?id=beacon&c=4
POST /pixel.gif?id=beacon&c=5
```

The incrementing `c=N` counter is a heartbeat sequence, and each POST carries a `session=` token that looks random - the data exfiltration payload. The User-Agent is `okhttp/3.12.0`, a Java HTTP client, not a browser - a strong IoC that this is automated malware-style traffic rather than a person.

Extract the full beacon including the exfiltrated token:

```
tshark -r /work/network.pcap -Y "http.request.uri contains \"pixel.gif\"" \
       -T fields -e frame.number -e http.request.uri -e http.file_data 2>/dev/null
```

---

8. Correlate everything into one timeline

Now merge the evidence from all phases into a single chronological timeline. The network capture gives you precise timestamps. Use tshark to print the time of each key event:

```
# time each HTTP request
tshark -r /work/network.pcap -Y "http.request" -T fields \
       -e frame.time -e http.request.method -e http.request.uri 2>/dev/null
```

Build the reconstruction table:

Time Phase Event Evidence T0	1	SSH brute force: 8 failed logins	auth.log, pcap
T1	2	HTTP recon: admin/backup/phpMyAdmin scans	httpd.log, pcap
T2	3a	Path traversal: /etc/passwd	httpd.log, pcap
T2	3b	SQLi on /login (`' OR 1=1--`)	httpd.log, pcap
T2	3c	Upload probe: shell.php to /upload	httpd.log, pcap
T3	4	C2 beacon: 5x POST /pixel.gif?id=beacon&c=N	httpd.log, pcap

---

9. Write the reconstruction report

A CPIA-compliant reconstruction report states, in order:

1. **Executive summary** - what happened, in two sentences.
2. **Timeline** - the phase-by-phase table from section 8.
3. **Evidence cited** - which log lines and which packets back each claim.
4. **Indicators of compromise (IoC)** - the attacker IP, the usernames, the beacon path and counter, the `okhttp/3.12.0` User-Agent.
5. **Impact and recommendation** - what was accessed, what to patch, what to monitor.

For the CPIA incident the conclusion is that the SSH brute force and web exploits did not achieve a successful login or web-shell upload, but the C2 beacon confirms an automated attacker established a persistence signalling channel - the incident is not contained until that beacon is understood.

Quick reference

```
# enter the analyst box
docker exec -it cpia-forensics bash

# baseline: what is in the capture
tshark -r /work/network.pcap -q -z io,phs 2>/dev/null
tshark -r /work/network.pcap -q -z endpoints,ip 2>/dev/null

# Phase 1 - SSH brute force
tshark -r /work/network.pcap -Y "tcp.port == 22" -T fields -e frame.number -e ip.src 2>/dev/null
grep -iE "Failed password|invalid user" /artifacts/auth.log 2>/dev/null

# Phase 2 - HTTP recon
tshark -r /work/network.pcap -Y "http.request" -T fields -e frame.number -e http.request.uri 2>/dev/
null

# Phase 3a - path traversal
tshark -r /work/network.pcap -Y "frame contains \"etc/passwd\"" -T fields -e frame.number 2>/dev/nul
l

# Phase 3b - SQLi
tshark -r /work/network.pcap -Y "frame contains \"OR 1=1\"" -T fields -e frame.number 2>/dev/null

# Phase 3c - upload probe
tshark -r /work/network.pcap -Y "frame contains \"shell.php\"" -T fields -e frame.number 2>/dev/null

# Phase 4 - C2 beacon
tshark -r /work/network.pcap -Y "http.request.uri contains \"pixel.gif\"" -T fields -e frame.number
-e http.request.uri -e http.file_data 2>/dev/null

# timeline with times
tshark -r /work/network.pcap -Y "http.request" -T fields -e frame.time -e http.request.uri 2>/dev/nu
ll
```

Pitfalls

- **Correlation is the point.** A pcap alone or a log alone tells only part of the story. The reconstruction is convincing when the same event is shown in both the packet capture and the host log.
- **Failed SSH logins look noisy.** Eight rapid failures in under two seconds is an automated brute force, not a tired user. Note the speed, not just the count.
- `/etc/passwd` **vs the web root.** Path traversal targets like `/files/../../../../etc/passwd` may be rewritten by the server; check the raw request in the capture, not the final response.
- **Beacons are subtle.** A tracking-pixel path with an incrementing counter and a random-looking session token is designed to look benign. The `okhttp/3.12.0` User-Agent (a Java client) is the giveaway that it is not a browser.
- **Timestamps.** The capture is authoritative for time. Keep the timezone consistent when you copy timestamps into the report.
- `2>/dev/null`**.** tshark emits SSH-dissector warnings for uncommon key-exchange algorithms; redirect stderr to keep the output clean.